

CAPSTONE PROJECT REPORT

Automated Testing of an E-Commerce Web Application

using Selenium WebDriver with Python

Capstone Project 1

Name: Puspendu Sekhar Das

Enrollment No.: 12023002022220

Stream: CST

Institute of Engineering and Management, Kolkata

Email Id: puspendusekhardas@gmail.com

Github Link: https://github.com/puspendudas01/Puthon_Automation_With_Selenium.git

Video Demonstration Link:

Abstract

Manual regression testing of an online shopping flow is repetitive and error-prone. This project delivers an automated test suite that simulates a customer purchasing a product on the public demo site automationexercise.com. Built with Selenium WebDriver, Python and Pytest using the Page Object Model, the suite launches a browser, registers and logs in a user, searches for a product, adds it to the cart, updates the quantity, and verifies the cart contents including unit price and line total. Test data is read from Excel and JSON files, popups and overlays are handled centrally, every step is captured as a screenshot, and an HTML execution report is generated on each run.

Keywords: Selenium WebDriver, Python, Pytest, Page Object Model, data-driven testing, test automation, e-commerce.

Contents

1. Introduction
2. Problem Statement and Objectives
3. Tools and Technologies
4. System Design
5. Implementation
6. Test Cases and Test Data
7. Execution Steps
8. Challenges and Solutions
9. Limitations and Future Scope
10. Conclusion
11. References

1. Introduction

E-commerce applications change frequently, and every release must be checked to ensure that core journeys such as login, search and checkout still work. Repeating these checks by hand is slow. Automated UI testing executes the same steps consistently in minutes and leaves evidence of what happened.

This capstone project applies Selenium WebDriver to automate the purchase journey of an online store. The application under test is <https://automationexercise.com>, a public site intended for practising test automation.

2. Problem Statement and Objectives

2.1 Problem statement

A customer wants to purchase a product from an e-commerce site. Design and implement an automated solution that verifies this journey end to end and reports the outcome.

2.2 Objectives

- Launch a browser and open the application.
- Log in with valid credentials.
- Search for a product and add it to the cart.

- Update the product quantity and verify the cart details.
- Capture screenshots at every important step.
- Read test data from Excel and JSON files.
- Handle popups, alerts and overlays if they appear.
- Generate an execution report.

2.3 Scope

The scope covers registration, login, search, add to cart, quantity update and cart verification. Payment, checkout and order placement are outside the scope of this version.

3. Tools and Technologies

Tool	Version used	Purpose
Python	3.14	Programming language
Selenium WebDriver	4.x	Browser automation; Selenium Manager downloads the driver automatically
Pytest	9.1	Test runner, fixtures, parametrisation
pytest-html	4.2	Self-contained HTML execution report
openpyxl	3.1	Reads test data from Excel
Google Chrome	153	Browser under automation
VS Code, Windows 11	-	Development environment

Hardware/software requirements: a Windows, macOS or Linux machine with Python 3.10 or later, Google Chrome, and an internet connection.

4. System Design

4.1 Architecture

The framework follows the Page Object Model. Each web page is represented by a class holding its locators and actions, while tests only describe the business flow. Configuration, test data and utilities are kept in separate modules.

Layer	Responsibility
Tests (tests/)	The end-to-end scenario, fixtures for browser and account, and report hooks.
Pages (pages/)	BasePage (waits, safe click, alerts, overlays), AuthPage, ProductPage, CartPage.
Utils (utils/)	Driver factory, Excel/JSON reader, screenshot helper.
Config (config/)	Base URL, browser, headless flag, timeouts and paths; overridable by environment variables.
Data (data/)	test_data.json (user profile) and test_data.xlsx (products).

Layer	Responsibility
Reports (reports/)	HTML report, JUnit XML, execution log and screenshots.

4.2 Folder structure

```
selenium_ecommerce_automation/
  main.py                      entry point (runs pytest)
  pytest.ini                   report + logging options
  requirements.txt
  config/config.py
  data/test_data.json, test_data.xlsx, create_test_data.py
  pages/base_page.py, auth_page.py, product_page.py, cart_page.py
  utils/driver_factory.py, data_reader.py, screenshot.py
  tests/conftest.py, test_purchase_flow.py
  reports/
```

generated on every run

4.3 Execution flow

main.py starts Pytest. A session fixture registers a throw-away user account. For each product row in the Excel sheet the test opens a new browser, logs in, clears the cart, searches, adds the product, changes the quantity, verifies the cart, takes screenshots and cleans up. After the session the account is deleted and the report is written.

5. Implementation

Requirement	How it is implemented
Launch browser	create_driver() in utils/driver_factory.py builds a Chrome (or Firefox) driver; headless mode via HEADLESS=true.
Login	AuthPage.login() fills the form using credentials from JSON/registration and asserts the 'Logged in as' banner.
Search product	ProductPage.search() types the Excel search term and waits for the results heading.
Add to cart	ProductPage.add_first_result_to_cart() clicks add to cart and waits for the confirmation modal.
Update quantity	The product details page is opened, the quantity field set to the Excel value, and the product added again; the site merges it into the same cart line.
Verify cart	CartPage.get_items() reads name, price, quantity and total; assertions check one line, name, quantity = old + extra, and total = price x quantity.
Screenshots	snap() saves a PNG per step into reports/screenshots and attaches it to the HTML report; a failure screenshot is added automatically.
Excel / JSON data	read_json() and read_excel() in utils/data_reader.py; one parametrised test per Excel row.

Requirement	How it is implemented
Popups / alerts	<code>accept_alert_if_present()</code> handles JavaScript alerts; consent dialogs and Google ad overlays are removed; JS-click fallback for intercepted clicks.
Execution report	pytest-html with <code>--self-contained-html</code> , plus JUnit XML and a log file, configured in <code>pytest.ini</code> .

5.1 Key design decisions

- Explicit waits only (`WebDriverWait` with expected conditions); no fixed sleeps.
- Locators use data-qa attributes and IDs wherever the site provides them.
- A new browser per test for isolation; one unique account per run so tests are repeatable.
- The cart is cleared before and after each scenario.
- All popup and overlay handling lives in `BasePage` so tests stay readable.

6. Test Cases and Test Data

6.1 Test cases

ID	Title	Description
TC01	Purchase flow - Blue Top	Login, search 'Blue Top', add to cart, add 2 more via product page, verify cart.
TC02	Purchase flow - Men Tshirt	Same flow with 3 additional units.
TC03	Purchase flow - Sleeveless Dress	Same flow with 1 additional unit.

6.2 Scenario steps and expected results

Step	Action	Expected result
1	Open home page	Title contains 'Automation Exercise'.
2	Log in	'Logged in as <name>' is visible.
3	Search product from Excel	At least one result; name matches expected name.
4	Add to cart	Confirmation modal; cart has one line, quantity 1.
5	Update quantity on product page	Modal appears; cart still one line.
6	Verify cart	Quantity = 1 + extra; price matches listing; total = price x quantity.
7	Clean up	Cart emptied, user logged out.

6.3 Test data

Excel file data/test_data.xlsx, sheet 'Products':

search_term	expected_name	extra_quantity
Blue Top	Blue Top	2
Men Tshirt	Men Tshirt	3
Sleeveless Dress	Sleeveless Dress	1

JSON file data/test_data.json holds the user profile (name, password, address and contact details) and the email prefix/domain used to generate a unique email for each run.

7. Execution Steps

```
python -m venv .venv                (optional)
.venv\Scripts\activate              (optional)
pip install -r requirements.txt
python main.py
```

Optional settings (environment variables): HEADLESS=true, BROWSER=firefox, BASE_URL, TEST_EMAIL and TEST_PASSWORD (use an existing account instead of registering a new one).

Outputs are written to the reports folder: execution_report.html, junit_results.xml, execution.log and the screenshots folder.

8. Challenges and Solutions

Challenge	Solution
Advertisements and overlays intercept clicks	Overlay-removal script and a JS-click fallback when Selenium reports ElementClickInterceptedException.
Slow page loads caused by ad scripts	Page-load strategy 'eager' and explicit waits for the exact elements needed.
Search-results heading not detected during the first execution	The site displays the heading in upper case ('SEARCHED PRODUCTS') and Selenium returns rendered text, so the wait was changed to a case-insensitive comparison.
No quantity box on the cart page	Quantity is set on the product details page; the expected value is computed from the quantity already in the cart.
Repeatable runs on a shared public site	Unique email per run, cart cleared before and after, throw-away account deleted at the end (a cleanup timeout is logged as a warning and does not fail tests).
Cookie dialogs and JavaScript alerts	Handled centrally in BasePage.

9. Limitations and Future Scope

9.1 Limitations

- Tests depend on a live public website, so page changes, ads or slow responses can affect results.
- Only Chrome and Firefox are supported; checkout and payment are not automated.

9.2 Future scope

- Add checkout and order confirmation steps.
- Add negative tests (invalid login, empty search, out-of-range quantity).
- Run tests in parallel with pytest-xdist and in CI with GitHub Actions.
- Add Allure reporting and cross-browser execution on Selenium Grid.

10. Conclusion

The project demonstrates an end-to-end automated test for an e-commerce purchase journey using Selenium WebDriver and Python. The Page Object Model, external test data, centralised popup handling and detailed reporting make the suite maintainable and easy to extend. The exercise provided practical experience in locator strategy, synchronisation, data-driven testing and test reporting, which are core skills for a test automation role.

11. References

- Selenium documentation: <https://www.selenium.dev/documentation/>
- Pytest documentation: <https://docs.pytest.org/>
- pytest-html documentation: <https://pytest-html.readthedocs.io/>
- openpyxl documentation: <https://openpyxl.readthedocs.io/>
- Application under test: <https://automationexercise.com>